NO-WAIT RESTAURANT

PROJECT DOCUMENTATION

A Web Application for Restaurant Reservations, QR Ordering, and Operations Management

Prepared as technical documentation for the NO-WAIT Restaurant project.

CHAPTER 1: PROJECT INTRODUCTION

1.1 Project Overview

NO-WAIT Restaurant is a web application that supports restaurant discovery, table reservations, ordering from a table QR page, and restaurant operations. The system consists of a React single-page frontend, a Node.js and Express API, and a SQLite database. It provides separate user experiences for customers and restaurant owners or staff.

The application aims to reduce friction in the dining process by allowing customers to review menu information, request a table, and place orders through a browser. The administration pages provide tools for reviewing reservations and orders and managing menu and table information.

1.2 Problem Statement

Restaurant visits can involve delays while guests choose a table, place an order, or obtain booking information. Restaurant staff also need a central place to review bookings, orders, menu entries, and table status. NO-WAIT brings these tasks together in one web application and makes availability and operational information easier to access.

1.3 Project Objectives

- Provide public pages for restaurant information and menu browsing.
- Support customer access to profile, reservations, and order history.
- Allow reservation requests using a date, arrival and departure times, party size, and table availability.
- Provide a QR ordering interface for a restaurant table.
- Provide owner dashboard pages for restaurant statistics and operational management.

1.4 Project Scope

The current repository contains frontend pages for public browsing, customer interactions, QR ordering, and owner administration. The backend provides JSON API endpoints and SQLite persistence. Some UI services also use sample data or browser local storage. Authentication and payment features are present as application flows, but their production readiness depends on real provider configuration and stronger server-side controls.

CHAPTER 2: SYSTEM REQUIREMENTS AND DESIGN

2.1 Functional Requirements

- Customers can browse the home page, restaurant information, and menu.
- Customers can sign in and access protected profile, reservation, and order pages.
- Customers can request a table reservation for a specified date, time interval, and guest count.
- Customers can open the QR ordering page for a table, select menu items, and use the cart and payment interface.
- Owners can view dashboard summaries and manage reservations, orders, menu items, and tables.
- The API can report service health and database status.

2.2 Non-Functional Requirements

- The frontend should support common desktop and mobile browser layouts.
- The API should return JSON responses with a success indicator and useful error messages.
- Reservation availability should account for overlapping time intervals and remaining table capacity.
- The database should preserve structured records for users, tables, menu items, reservations, orders, order items, and payments.
- Production deployment should use secure authentication, authorization, restricted network access, and environment-specific configuration.

2.3 System Architecture

The browser loads the React application. React Router maps the current URL to the appropriate page. Frontend service modules call the Express API over HTTP and exchange JSON. The API performs business logic and reads or writes records in SQLite. When the production frontend is built, Express can serve the generated static files as well as the API.

Layer	Technology	Responsibility
Presentation	React 18, Vite, React Router	Renders pages, navigation, forms, and interactive state.
Service/API client	JavaScript fetch wrapper	Calls backend endpoints and passes request data and customer identifier headers.

Layer	Technology	Responsibility
Application/API	Node.js and Express	Implements REST endpoints, validation and business operations; serves frontend build.
Data storage	SQLite	Stores relational restaurant, customer, reservation, order, and payment records.

2.4 Technology Stack

Technology	Use
React and Vite	Frontend components and development/build tooling.
React Router	Client-side routing and route composition.
Framer Motion	Interface animation.
Lucide React	Interface icon set.
Node.js and Express	Backend runtime and HTTP API framework.
SQLite	Embedded relational database.
Firebase Authentication client	Google sign-in client integration when configured.

CHAPTER 3: APPLICATION MODULES

3.1 Public Pages

The public site includes the home page, menu, about page, and how-it-works page. These pages introduce the restaurant and let visitors browse menu information. The corresponding paths include /, /menu, /about, and /how-it-works.

3.2 Customer Account and Profile

The customer sign-in flow is supported by the authentication context. Protected routes include booking, profile, order history, and reservation history. Customer state is used by the frontend to control access to these pages and to associate API requests with a customer record.

3.3 Reservation Management

The booking page collects reservation details such as date, arrival time, leaving time, guest count, and selected table. The backend availability logic checks active reservations for the selected date, determines which reservations overlap the requested interval, totals seats already reserved for each table, and compares the result with table capacity. The table-sharing behavior is enabled by default and can be disabled using TABLE_SHARING_ENABLED=false.

3.4 QR Ordering

The QR order page reads a table identifier from the URL query string, such as /qr-order?table=T01. The ordering interface provides menu selection, a cart, payment presentation, and order status feedback. The order service includes table identifier normalization and local-storage behavior for certain order history operations.

3.5 Owner Dashboard

The protected admin route loads the owner dashboard. The dashboard presents operational metrics and provides sections for reservations, orders, customers, menu items, analytics, and tables. Table updates include capacity and availability changes. Deleting a table is implemented as deactivation so prior records can be retained.

3.6 Frontend Code Organization

Path	Description
frontend/src/App.jsx	Defines public, customer-protected, and admin routes.
frontend/src/pages/	Public, customer, and admin page components.
frontend/src/components/	Reusable sections, dialogs, route guards, and UI components.
frontend/src/context/AuthContext.jsx	Authentication state and customer sign-in actions.
frontend/src/services/api.js	Backend API base URL and endpoint wrapper.
frontend/src/services/	Reservation, order, booking, and admin service logic.
frontend/src/config/firebase.js	Firebase client initialization and Google provider settings.
frontend/src/data/sampleMenu.js	Sample menu entries used by frontend flows.

CHAPTER 4: BACKEND API AND DATA DESIGN

4.1 API Conventions

Backend routes are prefixed with /api. Responses generally contain a success property and either a data value or an error message. The API client is defined in frontend/src/services/api.js. It currently uses http://localhost:5000/api as its base URL.

4.2 Main API Endpoints

Method	Endpoint	Purpose
GET	/api/health	Reports API and database health.
POST	/api/auth	Finds or creates a customer record.
GET, PATCH	/api/users/profile	Reads or updates a customer profile.
GET	/api/tables	Lists dining tables.
GET	/api/tables/available	Returns a basic table availability result.
GET	/api/tables/availability	Checks availability by date, time interval, and guest count.
GET	/api/menu	Lists menu items.
GET	/api/menu/:id	Returns one menu item.
POST, PATCH, DELETE	/api/menu and /api/menu/:id	Creates, updates, or removes menu items.
POST	/api/reservations	Creates a reservation.
GET	/api/reservations/my-reservations	Lists the current customer reservations.
DELETE	/api/reservations/:id	Cancels a reservation.
POST	/api/orders	Creates an order.
GET	/api/orders/my-orders	Lists the current customer orders.
GET, PATCH	/api/orders/:id	Reads an order or updates its status.
GET, POST	/api/payments/:paymentId and /verify	Reads payment details or runs verification flow.
GET	/api/admin/stats	Returns dashboard summary metrics.
GET, PATCH	/api/admin/reservations and /:id/...	Lists reservations and changes status or timing.

Method	Endpoint	Purpose
GET, PATCH	/api/admin/orders and /:id/status	Lists orders and changes order/payment status.
GET, POST, PATCH, DELETE	/api/admin/tables and /:id	Lists, creates, updates, or deactivates tables.

4.3 Database Schema

The database connection and schema setup are defined in backend/db.js. On initialization, the application creates the SQLite database file at backend/data/nowait.db if it does not exist and applies best-effort migrations for schema columns introduced in later project phases.

Entity	Purpose and principal relationships
users	Customer and staff identity/profile records.
restaurants	Restaurant records referenced by tables and transactional data.
tables	Table number, capacity, zone, owner availability, active state, and QR path.
menu_items	Restaurant menu item name, category, price, image, and availability.
reservations	Customer, table, date/time, guest count, status, and reservation payment fields.
orders	Customer, table, totals, order status, and payment state.
order_items	Items and quantities belonging to an order.
payments	Payment reference, amount, currency, method, and status.

CHAPTER 5: INSTALLATION AND CONFIGURATION

5.1 Prerequisites

- Node.js version 18 or later.
- npm package manager.
- A modern web browser for the frontend.

5.2 Start the Backend

From a terminal, change to the backend directory, install dependencies, and start the development server:

cd backend
npm install
npm run dev

The server listens at http://localhost:5000 by default. The PORT environment variable can be used to change the port. The database is initialized on first access.

5.3 Start the Frontend

In another terminal, change to the frontend directory, install dependencies, and start Vite:

cd frontend
npm install
npm run dev

Vite prints the local development address, usually http://localhost:5173. The frontend API service currently targets the backend at http://localhost:5000/api.

5.4 Firebase Configuration

The Firebase client reads these Vite environment variables:

VITE_FIREBASE_API_KEY
VITE_FIREBASE_AUTH_DOMAIN
VITE_FIREBASE_PROJECT_ID
VITE_FIREBASE_STORAGE_BUCKET
VITE_FIREBASE_MESSAGING_SENDER_ID
VITE_FIREBASE_APP_ID

Placeholder configuration is used when values are absent. A real Firebase project and enabled Google sign-in provider are required for functional Google authentication.

5.5 Build and Serve the Frontend

Run npm run build from the frontend directory to generate frontend/dist. Start the backend with npm start. Express serves the generated frontend files when the build exists, while also handling API routes.

5.6 Useful Development Commands

Command	Package directory	Purpose
npm run dev	frontend	Starts the Vite development server.
npm run build	frontend	Creates the production frontend build.
npm run preview	frontend	Previews the frontend production build.
npm run lint	frontend	Runs ESLint on the frontend.
npm run dev	backend	Starts the backend with Node watch mode.
npm start	backend	Starts the backend normally.

CHAPTER 6: SECURITY, LIMITATIONS, AND FUTURE WORK

6.1 Security Considerations

The current implementation is intended for development and demonstration. The following implementation details need attention before production use.

- Customer requests use an x-user-id header as an application identifier. This header alone does not prove the caller identity and must be replaced or protected by server-validated authentication.
- Some admin table API calls send a static x-admin-token from frontend code. A value embedded in client code is visible to users and is not a secure authorization mechanism. Admin authorization must be enforced on the server.
- The backend currently enables broad CORS. A deployed service should allow only trusted frontend origins.
- The frontend API origin is hard-coded to localhost. Deployed environments require configurable service URLs and HTTPS.
- Payment UI and verification endpoints do not by themselves establish a production payment integration. Use a provider and validate signed callbacks server-side.
- Firebase demo fallback values are placeholders and should not be used as production credentials.

6.2 Current Limitations

Parts of the frontend use sample menu content or local browser storage. The backend includes payment routes and database entities, but a production payment provider is not configured by the code reviewed for this documentation. Local development data may exist in backend/data/nowait.db and should be backed up before replacing the database.

6.3 Future Enhancements

- Implement secure session or token authentication and server-side role-based authorization.
- Move API URLs and configuration into environment-specific settings and restrict CORS.
- Integrate a payment provider with signed webhook verification and transaction reconciliation.
- Add automated API and frontend tests, request validation, error monitoring, and structured logs.
- Improve accessibility, responsive behavior, and usability through testing with customers and restaurant staff.

- Add deployment instructions, database backup and restore procedures, and operational monitoring.

6.4 Conclusion

NO-WAIT Restaurant demonstrates a full-stack application pattern that combines a React user interface, an Express REST API, and relational SQLite persistence. Its customer and owner features cover key restaurant activities, including menu browsing, table reservations, QR ordering, and operational management. Stronger authentication, deployment configuration, and real payment integration are important next steps for production readiness.

END OF DOCUMENT